

FinRAG

FINANCIAL DOCUMENT INTELLIGENCE SYSTEM

Retrieval-Augmented Generation (RAG) pipeline for Indian financial regulatory documents — RBI Circulars, SEBI Guidelines, and Corporate Annual Reports (FY 2024–25)

PyPDF2

LangChain

OpenAI Embeddings

FAISS

Streamlit

Project Report · FY 2024–25 · May 2026

NAVIGATION

Table of Contents

1	Project Overview & Objectives
2	System Architecture
3	Pipeline — Stage by Stage
4	Technology Stack
5	Chunking Strategy
6	Cost Estimation
7	Streamlit UI — Features
8	Sample Q&A; Outputs
9	Setup & Deployment Guide
10	Future Enhancements

Project Overview & Objectives

FinRAG is a Retrieval-Augmented Generation (RAG) system built to make Indian financial regulatory documents queryable using natural language. It processes RBI circulars, SEBI guidelines, and corporate annual reports, enabling analysts to ask questions and receive cited, accurate answers — without reading hundreds of pages.

#	Objective	Status
1	Parse 5–10 financial PDFs (RBI, SEBI, Bajaj Finserv)	Done
2	Clean and chunk text with LangChain (chunk=500, overlap=50)	Done
3	Embed chunks using OpenAI text-embedding-3-small	Done
4	Store vectors in FAISS for fast similarity search	Done
5	Generate cited answers with GPT-4o-mini	Done
6	Build Streamlit UI with Chat, Documents & About tabs	Done

System Architecture

The pipeline follows a standard RAG architecture with financial-domain customizations for Indian regulatory documents:

■ PDF Ingestion	5–10 financial PDFs loaded from pdfs/ directory
■ Text Extraction	PyPDF2 extracts raw text page-by-page with metadata
■ Text Cleaning	Regex removes headers, footers, page numbers, artifacts
>■ Chunking	LangChain splits text: chunk=500 chars, overlap=50
■ Embedding	OpenAI text-embedding-3-small → 1536-dim vectors
■■ Vector Storage	FAISS IndexFlatIP with L2 normalization for cosine search
■ Retrieval	Query embedded → top-K chunks retrieved by similarity
■ Generation	GPT-4o-mini produces cited answer from retrieved context

03

Pipeline — Stage by Stage

Stage 1 — PDF Parsing

File: parse_pdfs.py

```
reader = PyPDF2.PdfReader(filepath)
for page in reader.pages:
    text = page.extract_text()
```

Stage 2 — Text Cleaning

File: clean_text.py

```
text = re.sub(r"\n{3,}", "\n\n", raw_text)
text = re.sub(r"- \n(\w)", r"\1", text)
text = re.sub(r"^\s*\d{1,4}\s*$", "", text, flags=re.MULTILINE)
```

Stage 3 — Chunking

File: chunk_text.py

```
splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=50,
    separators=["\n\n", "\n", ". ", " ", "", ""])
```

Stage 4 — Embedding

File: embed_store.py

```
response = client.embeddings.create(
    model="text-embedding-3-small",
    input=batch # up to 50 chunks per call
)
```

Stage 5 — FAISS Store

File: embed_store.py

```
faiss.normalize_L2(matrix)
index = faiss.IndexFlatIP(1536)
index.add(matrix)
faiss.write_index(index, "vector_store/index.faiss")
```

Stage 6 — RAG Query

File: `rag_query.py`

```
scores, indices = index.search(query_vec, top_k)
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[system_prompt, context + question]
)
```

Technology Stack

Component	Library / Service	Version	Purpose
PDF Parsing	PyPDF2	3.x	Extract text from PDF pages
Text Splitting	LangChain TextSplitter	0.3.x	Chunk text with overlap
Embeddings	text-embedding-3-small	OpenAI	1536-dim semantic vectors
Vector Store	FAISS IndexFlatIP	1.7.x	Cosine similarity search
LLM Generation	GPT-4o-mini	OpenAI	Cited answer generation
UI Framework	Streamlit	1.x	Web app with tabs & chat
Language	Python	3.10+	Core implementation

Chunking Strategy

RecursiveCharacterTextSplitter attempts splits in order: paragraph → newline → sentence → space → character. This preserves semantic boundaries and reduces mid-sentence cuts.

Setting	Value	Reason
chunk_size	500	Fits OpenAI embedding limits; granular enough for precise retrieval
chunk_overlap	50	Prevents context loss at boundaries — 10% of chunk size
Separator 1	\n\n	Paragraph breaks — highest semantic boundary
Separator 2	\n	Line breaks — section headers, list items
Separator 3	.	Sentence endings — natural clause boundaries
Separator 4		Word boundary fallback
Separator 5	""	Hard character cut — last resort only

Cost Estimation (FY 2024–25)

Based on 8 PDFs averaging 40 pages each (~2,560 chunks) and 200 queries/month:

Item	Model	Rate	Est. Cost
One-time embedding	text-embedding-3-small	\$0.02 / 1M tokens	~\$0.01
Per query — input tokens	gpt-4o-mini	\$0.15 / 1M tokens	~\$0.001
Per query — output tokens	gpt-4o-mini	\$0.60 / 1M tokens	~\$0.001
200 queries / month	gpt-4o-mini	—	~\$0.25
Total first month	Combined	—	~\$0.26
Total in INR (approx)	Combined	@ ₹83/USD	~₹22

Streamlit UI — Features

■ Chat Tab	Streaming answers from GPT-4o-mini with real-time token display. Suggested question buttons for quick start. Expandable source citations showing exact chunk, page, and similarity score for every answer.
■ Documents Tab	Browse all indexed documents. Select any PDF to see chunk count, page count, and average chunk size. Filter by page number and inspect individual chunk text.
■ About Tab	Interactive pipeline diagram, live cost estimator, setup checklist with real-time status detection, and FAQ accordion with 6 common questions answered.
■ Sidebar Controls	Live API key input, top-k slider (1–10), model selector (gpt-4o-mini / gpt-4o / gpt-3.5-turbo), knowledge base stats, session query counter, and clear chat button.
■ Dark Theme	Professional financial aesthetic using JetBrains Mono + Syne fonts, #0D1117 background, #00C49A accent — matching GitHub's dark mode palette.

Sample Q&A; Outputs

A: According to RBI Circular 2024 (p.3), all digital lenders must disclose the Annual Percentage Rate (APR) upfront. Loan repayments must flow directly through the regulated entity's bank account (p.5). A minimum 3-day cooling-off period must be offered to all borrowers before loan acceptance.

Source: rbi_circular_2024.pdf · p.3, p.5 · Score: 0.921

A: Per the Bajaj Finserv Annual Report FY2024-25 (p.12), the consolidated total income stood at **₹1,14,232** crore, representing a 28% year-on-year growth driven primarily by insurance premium income and AUM expansion in the lending segment.

Source: bajaj_finserv_annual_2025.pdf · p.12 · Score: 0.894

A: SEBI Guidelines 2024 (p.7) mandate that brokers collect upfront margins for all intraday and delivery trades. Peak margin shortfall attracts a penalty of 0.5% per day. From August 2024, reporting frequency moved to 4 snapshots daily.

Source: sebi_guidelines_2024.pdf · p.7 · Score: 0.878

Setup & Deployment Guide

Step 1: Install dependencies

```
pip install PyPDF2 langchain langchain-text-splitters openai faiss-cpu
pip install numpy tiktoken streamlit reportlab
```

Step 2: Set OpenAI API key

```
Windows: set OPENAI_API_KEY=sk-...
Mac/Linux: export OPENAI_API_KEY=sk-...
```

Step 3: Add PDF files

```
Copy your PDFs into the pdfs/ folder.
Supported: RBI circulars, SEBI guidelines, annual reports (FY24-25)
```

Step 4: Run the full pipeline

```
python main.py
# Parses → Cleans → Chunks → Embeds → Stores in FAISS
```

Step 5: Launch Streamlit UI

```
streamlit run app.py
# Opens at http://localhost:8501
```

Step 6: Deploy to Streamlit Cloud (free)

1. Push project to GitHub
2. Go to share.streamlit.io
3. Connect repo → set OPENAI_API_KEY as secret
4. Deploy – get a public URL instantly

Future Enhancements

Enhancement	Benefit	Effort
OCR for scanned PDFs	Parse image-only RBI documents	Medium
Pinecone / Chroma vector DB	Cloud-hosted, persistent storage	Low
Multi-document comparison	Compare SEBI vs RBI on same topic	Medium
Hindi language support	Query in Hindi, answer in Hindi	High
PDF upload in UI	User uploads PDFs via Streamlit	Low
Answer confidence score	Show how certain the model is	Medium
Email alert on new circulars	Auto-fetch new RBI/SEBI PDFs	High